

Chess Concept Library

Combined concept/principle list from T. McGrath, A. Kapishnikov, N. Tomasev, A. Pearce, M. Wattenberg, D. Hassabis, B. Kim, U. Paquet, V. Kramnik, "Acquisition of Chess Knowledge in AlphaZero" (PNAS 2022; arXiv:2111.09259), Supplementary Information Tables S1, S2 and S3.

Purpose: structured chess knowledge to supply to an LLM alongside the position (e.g. FEN) and a piece-by-piece explanation. Concept names are the probe labels used in the paper; descriptions are condensed and lightly edited from the paper's tables and from Stockfish 8's evaluation semantics.

Naming conventions

Fully qualified names follow `<concept>_<side>` (custom concepts) or `<concept>_<side>_<phase>` (Stockfish concepts).

Side suffixes:

`mine` = the side to move. AlphaZero always represents the position from the mover's perspective, so sides are "mine"/"opponent" rather than White/Black.

`opponent` = the other side.

`t` = total, i.e. the difference between the two sides (Stockfish concepts).

`diff` = mine minus opponent (custom concepts).

Phase suffixes (Stockfish concepts only):

`mg` = middlegame value.

`eg` = endgame value.

`ph` = phased value: a weighted sum of the `mg` and `eg` values based on the actual phase of the position.

Squares: in the capture concepts, the squares (`d1`, `d2`, `d3`, `e1`, `e2`, `e3`, `g5`, `b5`) are named as if the side in question were playing White (again because the board is always oriented toward the side to move).

Value types: `has_*`, `in_check`, `can_*` and `capture_*` concepts are boolean; `num_*`, the custom `material` and `pawns_on_7th_rank` are integer counts; all Stockfish concepts are real-valued evaluation scores.

Counts

Table S1 (Stockfish 8 evaluation): 13 families -> 93 concepts. Derivation: 4 total-only families (material, imbalance, pawns, total) x 3 phases + 9 per-side families x 3 sides x 3 phases = 12 + 81 = 93. The 13 families and the total-only behaviour of material, imbalance, pawns and total match Stockfish 8's own evaluation trace (`src/evaluate.cpp`, tag `sf_8`). The SI figures plot only 80 of the 93 (queens, `material_t_ph` and `space*_eg` are defined but never plotted).

Table S2 (custom): 27 families -> 78 concepts. Note: `capture_happens_next_move_on_<sq>` carries no `[m]o` suffix in Table S2, so it contributes 8 concepts (one per square).

Table S3 (custom, pawns): 14 families -> 36 concepts. `num_double_pawn_files` is listed with suffixes `[t|m]o[diff]` (4 variants).

Custom total (S2 + S3): 114 enumerated concepts. The PNAS main text states "our own implementations of 116 concepts", a gap of 2 that the SI tables do not resolve; the most likely cause is a side suffix omitted in print for one of the single-variant rows (e.g. `in_check` or `has_mate_threat`).

Grand total enumerated here: $93 + 78 + 36 = 207$ concepts across 54 families (the paper's implied total is $93 + 116 = 209$).

material appears in BOTH S1 (Stockfish phase-dependent centipawn score) and S2 (simple 1/3/3/5/9 count); both are kept, distinguished by their suffixes.

PART 1 - Stockfish 8 evaluation concepts (Table S1) - 93 concepts

Each family below is a component of Stockfish 8's static evaluation function, exposed through its public API (the same breakdown printed by the engine's `eval` trace).

1. material [sides: t | phases: mg, eg, ph]

Material score. Every piece on the board has a predefined value that changes with the phase of the game, and each piece is valued in isolation, independently of the other pieces (contrast with imbalance).

Variants (3): `material_t_mg`, `material_t_eg`, `material_t_ph`

2. imbalance [sides: t | phases: mg, eg, ph]

Second-order material score: the value of each piece is adjusted with respect to all other pieces on the board, favouring or penalising particular piece-count combinations (e.g. a bishop pair is preferred over bishop plus knight). Computed as a quadratic form of the piece counts with a preset weight matrix.

Variants (3): `imbalance_t_mg`, `imbalance_t_eg`, `imbalance_t_ph`

3. pawns [sides: t | phases: mg, eg, ph]

Pawn-structure evaluation: isolated, doubled, connected, backward, blocked, weak pawns and similar features.

Variants (3): `pawns_t_mg`, `pawns_t_eg`, `pawns_t_ph`

4. knights [sides: mine, opponent, t | phases: mg, eg, ph]

Knight evaluation; e.g. extra points for knights occupying outposts protected by pawns.

Variants (9): `knights_mine_mg`, `knights_mine_eg`, `knights_mine_ph`, `knights_opponent_mg`, `knights_opponent_eg`, `knights_opponent_ph`, `knights_t_mg`, `knights_t_eg`, `knights_t_ph`

5. bishops [sides: mine, opponent, t | phases: mg, eg, ph]

Bishop evaluation; e.g. bishops standing on squares of the same colour as their own pawns are penalised. (The paper's probe labels use the singular bishop.)

Variants (9): `bishop_mine_mg`, `bishop_mine_eg`, `bishop_mine_ph`, `bishop_opponent_mg`, `bishop_opponent_eg`, `bishop_opponent_ph`, `bishop_t_mg`, `bishop_t_eg`, `bishop_t_ph`

6. rooks [sides: mine, opponent, t | phases: mg, eg, ph]

Rook evaluation; e.g. rooks occupying open or semi-open files are valued more highly.

Variants (9): `rooks_mine_mg`, `rooks_mine_eg`, `rooks_mine_ph`, `rooks_opponent_mg`, `rooks_opponent_eg`, `rooks_opponent_ph`, `rooks_t_mg`, `rooks_t_eg`, `rooks_t_ph`

7. queens [sides: mine, opponent, t | phases: mg, eg, ph]

Queen evaluation; e.g. queens that are subject to a relative pin or a discovered attack are penalised.

Variants (9): queens_mine_mg, queens_mine_eg, queens_mine_ph, queens_opponent_mg, queens_opponent_eg, queens_opponent_ph, queens_t_mg, queens_t_eg, queens_t_ph

8. mobility [sides: mine, opponent, t | phases: mg, eg, ph]

Piece-mobility score, depending on the number of squares attacked by the side's pieces.

Variants (9): mobility_mine_mg, mobility_mine_eg, mobility_mine_ph, mobility_opponent_mg, mobility_opponent_eg, mobility_opponent_ph, mobility_t_mg, mobility_t_eg, mobility_t_ph

9. king_safety [sides: mine, opponent, t | phases: mg, eg, ph]

Complex king-safety term: number and type of pieces attacking the squares around the king, pawn-shelter strength, number of pawns around the king, penalties for a king on a pawnless flank, and similar features.

Variants (9): king_safety_mine_mg, king_safety_mine_eg, king_safety_mine_ph, king_safety_opponent_mg, king_safety_opponent_eg, king_safety_opponent_ph, king_safety_t_mg, king_safety_t_eg, king_safety_t_ph

10. threats [sides: mine, opponent, t | phases: mg, eg, ph]

Threats against pieces: whether a pawn can safely advance and attack a higher-value enemy piece, hanging pieces, possible x-ray attacks by rooks, and similar features.

Variants (9): threats_mine_mg, threats_mine_eg, threats_mine_ph, threats_opponent_mg, threats_opponent_eg, threats_opponent_ph, threats_t_mg, threats_t_eg, threats_t_ph

11. passed_pawns [sides: mine, opponent, t | phases: mg, eg, ph]

Bonuses for passed pawns; the closer a pawn is to the promotion rank, the larger the bonus.

Variants (9): passed_pawns_mine_mg, passed_pawns_mine_eg, passed_pawns_mine_ph, passed_pawns_opponent_mg, passed_pawns_opponent_eg, passed_pawns_opponent_ph, passed_pawns_t_mg, passed_pawns_t_eg, passed_pawns_t_ph

12. space [sides: mine, opponent, t | phases: mg, eg, ph]

Space evaluation: number of safe squares available for minor pieces on the central four files, on ranks 2 to 4.

Variants (9): space_mine_mg, space_mine_eg, space_mine_ph, space_opponent_mg, space_opponent_eg, space_opponent_ph, space_t_mg, space_t_eg, space_t_ph

13. total [sides: t | phases: mg, eg, ph]

The total evaluation of the position, encapsulating all of the concepts above.

Variants (3): total_t_mg, total_t_eg, total_t_ph

PART 2 - Custom concepts (Table S2) - 78 concepts

Implemented programmatically by the authors (not taken from Stockfish 8's API).

14. pawn_fork [sides: mine, opponent]

True if the side has a pawn that attacks two enemy pieces of higher value (knight, bishop, rook, queen or king) and the pawn is not pinned.

`Variants (2): pawn_fork_mine, pawn_fork_opponent`

15. knight_fork [sides: mine, opponent]

True if the side has a knight that attacks two enemy pieces of higher value (rook, queen or king) and the knight is not pinned.

`Variants (2): knight_fork_mine, knight_fork_opponent`

16. bishop_fork [sides: mine, opponent]

True if the side has a bishop that attacks two enemy pieces of higher value (rook, queen or king) and the bishop is not pinned.

`Variants (2): bishop_fork_mine, bishop_fork_opponent`

17. rook_fork [sides: mine, opponent]

True if the side has a rook that attacks two enemy pieces of higher value (queen or king) and the rook is not pinned.

`Variants (2): rook_fork_mine, rook_fork_opponent`

18. has_pinned_pawn [sides: mine, opponent]

True if the side has a pawn that is pinned to its own king.

`Variants (2): has_pinned_pawn_mine, has_pinned_pawn_opponent`

19. has_pinned_knight [sides: mine, opponent]

True if the side has a knight that is pinned to its own king.

`Variants (2): has_pinned_knight_mine, has_pinned_knight_opponent`

20. has_pinned_bishop [sides: mine, opponent]

True if the side has a bishop that is pinned to its own king.

`Variants (2): has_pinned_bishop_mine, has_pinned_bishop_opponent`

21. has_pinned_rook [sides: mine, opponent]

True if the side has a rook that is pinned to its own king.

`Variants (2): has_pinned_rook_mine, has_pinned_rook_opponent`

22. has_pinned_queen [sides: mine, opponent]

True if the side has a queen that is pinned to its own king.

`Variants (2): has_pinned_queen_mine, has_pinned_queen_opponent`

23. material (custom) [sides: mine, opponent, diff]

Simple material count: $(\#pawns) + 3 * (\#knights) + 3 * (\#bishops) + 5 * (\#rooks) + 9 * (\#queens)$.

Distinct from the Stockfish material concept in Part 1.

`Variants (3): material_mine, material_opponent, material_diff`

24. num_pieces [sides: mine, opponent, diff]

Number of pieces that the side has.

`Variants (3): num_pieces_mine, num_pieces_opponent, num_pieces_diff`

25. in_check [single]

True if the side to move is in check.

Variants (1): in_check

26. has_bishop_pair [sides: mine, opponent]

True if the side has a pair of bishops.

Variants (2): has_bishop_pair_mine, has_bishop_pair_opponent

27. has_connected_rooks [sides: mine, opponent]

True if the side has connected rooks.

Variants (2): has_connected_rooks_mine, has_connected_rooks_opponent

28. has_control_of_open_file [sides: mine, opponent]

True if the side controls an open file (with its rooks and/or queen).

Variants (2): has_control_of_open_file_mine, has_control_of_open_file_opponent

29. has_mate_threat [single]

True if the opponent could mate the side to move in a single move if the turn were passed to the opponent (i.e. the mover is under a mate-in-one threat).

Variants (1): has_mate_threat

30. has_check_move [sides: mine, opponent]

True if the side has a move that gives check to the enemy king.

Variants (2): has_check_move_mine, has_check_move_opponent

31. can_capture_queen [sides: mine, opponent]

True if the side can capture the opponent's queen.

Variants (2): can_capture_queen_mine, can_capture_queen_opponent

32. num_king_attacked_squares [sides: mine, opponent, diff]

Number of squares around the enemy king that the given side attacks; occupied squares can be included.

Variants (3): num_king_attacked_squares_mine, num_king_attacked_squares_opponent, num_king_attacked_squares_diff

33. has_contested_open_file [single]

True if an open file is occupied simultaneously by a rook and/or queen of both colours.

Variants (1): has_contested_open_file

34. has_right_bc_ha_promotion [sides: mine, opponent]

True if the side has (1) a passed pawn on the a- or h-file and (2) a bishop whose square colour matches the promotion square of that pawn (the "right-coloured bishop" for a rook-pawn promotion).

Variants (2): has_right_bc_ha_promotion_mine, has_right_bc_ha_promotion_opponent

35. num_scb_pawns_same_side [sides: mine, opponent, diff]

Number of the side's own pawns that occupy squares of the same colour as its own bishop. Only applicable when the side has a single bishop.

Variants (3): num_scb_pawns_same_side_mine, num_scb_pawns_same_side_opponent, num_scb_pawns_same_side_diff

36. num_ocb_pawns_same_side [sides: mine, opponent, diff]

Number of the side's own pawns that occupy squares of the opposite colour to its own bishop. Only applicable when the side has a single bishop.

Variants (3): num_ocb_pawns_same_side_mine, num_ocb_pawns_same_side_opponent, num_ocb_pawns_same_side_diff

37. num_scb_pawns_other_side [sides: mine, opponent, diff]

Number of the opponent's pawns that occupy squares of the same colour as the side's own bishop. Only applicable when the side has a single bishop.

Variants (3): num_scb_pawns_other_side_mine, num_scb_pawns_other_side_opponent, num_scb_pawns_other_side_diff

38. num_ocb_pawns_other_side [sides: mine, opponent, diff]

Number of the opponent's pawns that occupy squares of the opposite colour to the side's own bishop. Only applicable when the side has a single bishop.

Variants (3): num_ocb_pawns_other_side_mine, num_ocb_pawns_other_side_opponent, num_ocb_pawns_other_side_diff

39. capture_possible_on_<sq> [sides: mine, opponent | squares: d1, d2, d3, e1, e2, e3, g5, b5]

True if the side can capture a piece on the given square. Squares are named as if the side were playing White.

Variants (16): capture_possible_on_d1_mine, capture_possible_on_d1_opponent, capture_possible_on_d2_mine, capture_possible_on_d2_opponent, capture_possible_on_d3_mine, capture_possible_on_d3_opponent, capture_possible_on_e1_mine, capture_possible_on_e1_opponent, capture_possible_on_e2_mine, capture_possible_on_e2_opponent, capture_possible_on_e3_mine, capture_possible_on_e3_opponent, capture_possible_on_g5_mine, capture_possible_on_g5_opponent, capture_possible_on_b5_mine, capture_possible_on_b5_opponent

40. capture_happens_next_move_on_<sq> [squares: d1, d2, d3, e1, e2, e3, g5, b5 | no side suffix in Table S2]

True if a capture of a piece on the given square actually happened, according to the game record - a game-trajectory label rather than a static board property. Squares are named as if the side were playing White.

Variants (8): capture_happens_next_move_on_d1, capture_happens_next_move_on_d2, capture_happens_next_move_on_d3, capture_happens_next_move_on_e1, capture_happens_next_move_on_e2, capture_happens_next_move_on_e3, capture_happens_next_move_on_g5, capture_happens_next_move_on_b5

PART 3 - Custom pawn-structure concepts (Table S3) - 36 concepts

Implemented programmatically by the authors (not taken from Stockfish 8's API).

41. num_double_pawn_files [sides: t, mine, opponent, diff]

Number of files that carry more than one pawn of the side's colour (files with doubled pawns).

Variants (4): num_double_pawn_files_t, num_double_pawn_files_mine, num_double_pawn_files_opponent, num_double_pawn_files_diff

42. has_double_pawn [sides: mine, opponent]

True if the side has doubled pawns, i.e. some file contains more than one of its pawns.

Variants (2): has_double_pawn_mine, has_double_pawn_opponent

43. num_isolated_pawns [sides: mine, opponent, diff]

Number of the side's pawns that have no friendly pawns on the files to their left and right.

Variants (3): num_isolated_pawns_mine, num_isolated_pawns_opponent, num_isolated_pawns_diff

44. has_isolated_pawn [sides: mine, opponent]

True if the side has a pawn with no friendly pawns on the adjacent files.

Variants (2): has_isolated_pawn_mine, has_isolated_pawn_opponent

45. has_pawn_on_7th_rank [sides: mine, opponent]

True if the side has a pawn that has reached the 7th rank (from its own perspective; one step from promotion).

Variants (2): has_pawn_on_7th_rank_mine, has_pawn_on_7th_rank_opponent

46. pawns_on_7th_rank [sides: mine, opponent, diff]

Number of the side's pawns that have reached the 7th rank.

Variants (3): pawns_on_7th_rank_mine, pawns_on_7th_rank_opponent, pawns_on_7th_rank_diff

47. has_passed_pawn [sides: mine, opponent]

True if the side has a pawn with no opposing pawns able to prevent it from advancing to the eighth rank.

Variants (2): has_passed_pawn_mine, has_passed_pawn_opponent

48. num_passed_pawns [sides: mine, opponent, diff]

Number of passed pawns the side has.

Variants (3): num_passed_pawns_mine, num_passed_pawns_opponent, num_passed_pawns_diff

49. has_protected_passed_pawn [sides: mine, opponent]

True if the side has a passed pawn that is protected by one of its own pawns.

Variants (2): has_protected_passed_pawn_mine, has_protected_passed_pawn_opponent

50. num_protected_passed_pawns [sides: mine, opponent, diff]

Number of protected passed pawns the side has.

Variants (3): num_protected_passed_pawns_mine, num_protected_passed_pawns_opponent, num_protected_passed_pawns_diff

51. num_pawn_islands [sides: mine, opponent, diff]

Number of pawn islands (maximal groups of the side's pawns on consecutive files, separated by files without its pawns).

Variants (3): num_pawn_islands_mine, num_pawn_islands_opponent, num_pawn_islands_diff

52. has_iqp [sides: mine, opponent]

True if the side has an isolated queen's pawn (an isolated pawn on the d-file).

Variants (2): has_iqp_mine, has_iqp_opponent

53. has_connected_passed_pawns [sides: mine, opponent]

True if the side has two or more passed pawns on adjacent files.

Variants (2): has_connected_passed_pawns_mine, has_connected_passed_pawns_opponent

54. num_connected_passed_pawns [sides: mine, opponent, diff]

Number of connected passed pawns the side has.

Variants (3): num_connected_passed_pawns_mine, num_connected_passed_pawns_opponent,
num_connected_passed_pawns_diff